



MoveIt

Motion Planning Framework for ROS 2

Presented by **Gaurav Phatak**

What is MoveIt?



MoveIt 2 is the leading **open-source motion planning framework** for robotic manipulators, built natively on ROS 2. It bridges high-level planning and low-level robot control — enabling safe, intelligent movement in both simulation and on physical hardware.

→ **Open-Source**
Community-driven, actively maintained

→ **ROS 2 Native**
Designed for the modern ROS 2 ecosystem

→ **Simulation & Real Robots**
Works in Gazebo and on physical hardware

Why MoveIt?



Automated Motion Planning

Generates collision-free trajectories automatically, removing the need for manual path scripting.



Collision Avoidance

Continuously monitors the environment to prevent contact with obstacles or self-collision.



Inverse Kinematics

Solves joint configurations required to reach a desired end-effector pose.



Path Planning

Employs algorithms like OMPL and RRTConnect to compute optimal paths.



Robot Control Integration

Seamlessly interfaces with `ros2_control` for real-time execution on hardware.

Movelt Architecture

Movelt operates as a layered middleware stack — translating user intent into physical robot motion through a chain of well-defined components.

User

Defines desired goal pose

Movelt

Plans collision-aware trajectory

RViz

Visualizes scene and sends commands

ROS 2

Handles topic and service comms



Each layer communicates via ROS 2 topics and services, ensuring a clean separation between planning logic and hardware execution.

End-to-End Workflow

From robot definition to physical motion — a structured seven-step process.

01

Create Robot Description

Define the robot using a URDF file

02

Configure MoveIt

Use the MoveIt Setup Assistant

03

Generate Config Package

Output SRDF, launch files, and settings

04

Launch MoveIt

Start the MoveIt node alongside ROS 2

05

Select Goal Pose

Choose a target in RViz

06

Plan Trajectory

Compute a safe, collision-free path

07

Execute Motion

Send commands to the robot controller

Key Components

URDF

Unified Robot Description Format — defines links, joints, and collision geometry.

SRDF

Semantic Robot Description — adds groups, end-effectors, and collision rules.

RViz

ROS visualization tool — displays robot state and allows interactive goal selection.

Movelt Setup Assistant

GUI tool that generates the Movelt configuration package from a URDF.

ros2_control

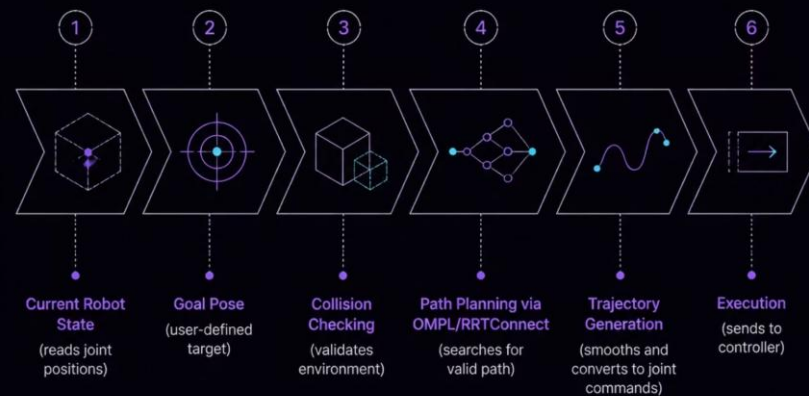
ROS 2 framework for hardware abstraction and controller management.

Planning Scene

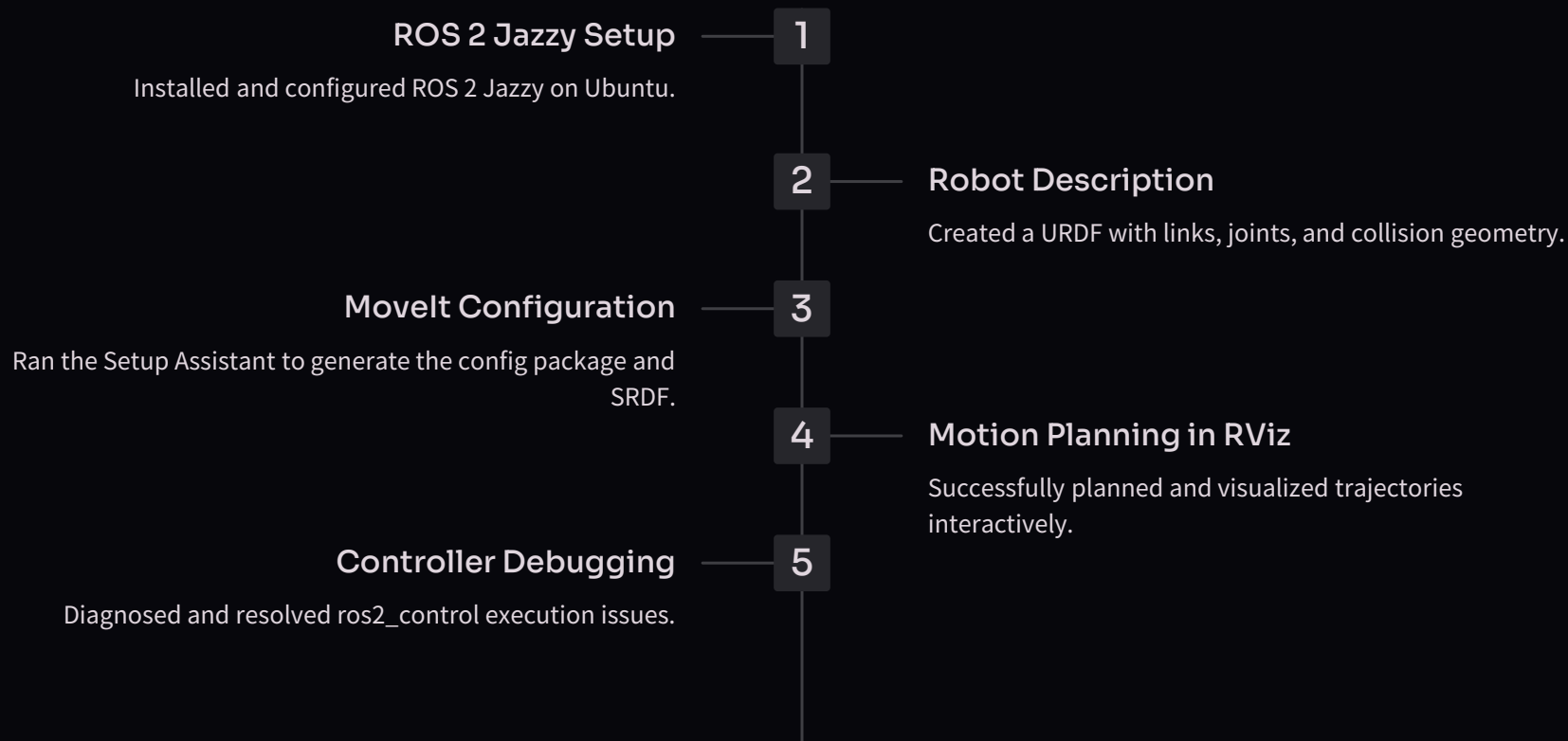
Real-time world model including robot state, obstacles, and collision objects.

Motion Planning Process

Movelt's planning pipeline transforms a goal request into an executable trajectory in milliseconds. The process begins with the **current robot state** and the **desired goal pose**, then performs **collision checking** against the planning scene. Algorithms like **OMPL** and **RRTConnect** search for a valid path, which is smoothed and converted into a **joint-space trajectory** before being sent to the controller for execution.



My Implementation



Challenges & Lessons Learned

Package Dependencies

Resolving version mismatches across the ROS 2 ecosystem.

Controller Configuration

Correctly wiring `ros2_control` YAML files for hardware interfaces.

URDF Joint Limits

Incorrect limits caused planning failures and unexpected behavior.

ROS Ecosystem Learning

Navigating nodes, topics, parameters, and launch files.

Key Takeaway

Every error message and debugging session is an opportunity to deepen your understanding of the system.

Problem-solving and debugging are not obstacles — they are the core of robotics development.

- ❏ Each challenge resolved builds stronger intuition for designing robust robotic systems.



Conclusion

MoveIt Simplifies Planning

Abstracts complex algorithms into a usable framework.

ROS 2 Enables Communication

Provides the backbone for distributed robot systems.

RViz Visualizes Everything

Interactive visualization for planning and debugging.

Controllers Execute Safely

`ros2_control` handles real-time hardware execution.

Thank you — Questions?